

Bayesian Analysis of BTC-USDT Trades

Team:

- Shaik Sattar Saif (MA23BTECH11023)
- Chaitanya Nemmani (MA23BTECH11008)

You can find all of our codes (including codes relevant to project 1) here:

[GitHub link \(https://github.com/Zen-Nightshade/Crypto-Trade-Analysis\)](https://github.com/Zen-Nightshade/Crypto-Trade-Analysis)

The relevant codes are mentioned in this report wherever necessary.

Abstract

This project advances Bayesian analysis of the BTC/USDT trade dataset from Binance by extending and refining the modeling framework developed in the previous phase (Project-1). We have strengthened Model comparison for the Joint Normal model through the implementation of simple and composite Bayes factors for hypotheses concerning the mean parameter μ . Bayesian regression on log-returns is performed using a Gibbs sampling scheme to obtain posterior estimates for regression coefficients and noise variance. To model trade counts more effectively, we introduce a hierarchical model based on a Negative Binomial formulation, and we estimate its parameters using a Metropolis–Hastings (MH) sampler. We also revisit the LogNormal–Poisson model and apply MH to fit it, allowing us to test a more flexible sampling approach. These additions strengthen the overall Bayesian framework and help us better understand patterns and variability in cryptocurrency trading.

Introduction

In the previous phase of the project (Project-1), we analysed BTC/USDT trades dataset from Binance using Bayesian models for both trade count and trade volumes, establishing a basic framework for posterior estimation and model evaluation. This has provided an initial insight into how trading activity behaves over time.

In this second phase, we extend that framework with more advanced Bayesian techniques, by introducing stronger model comparison tools, performing Bayesian regression on log-returns, and developed more flexible hierarchical Negative Binomial model for modeling trade count. These methods are implemented using Gibbs sampling and Metropolis Hasting algorithms, allowing for a richer posterior inference.

Since the dataset is time-ordered, we also study temporal dependence through lag-based modelling. As our previous report revealed high values of auto-correlation at small lag values, indicating the recent trade data strongly influences the current behavior. Motivated by this, we incorporate lagged observations into the likelihood i.e. at any time t we consider the data from $t - lag$ to $t - 1$ for the likelihood to examine how model performance changes as the time passes.

Overall this phase expands the earlier framework with more advanced Bayesian methods and a clearer treatment of time dependence.

Bayes Factor Analysis

Bayesian Log-Volume Modeling

In the first phase of the project, we used a Joint-Normal (both μ, σ are unknown) model to predict the logarithm of total trade cost at every minute. We now extend that work by applying the model to compute **Bayes factors** for hypothesis testing.

We analyze the transformed variable

$$X_t = \log(V_t)$$

where V_t is the total trade cost in minute t . We use the log transformation because raw trade volumes tend to be highly variable and positively skewed. Working in log-space stabilizes variance and makes the distribution more suitable for Gaussian modeling.

We use the **Joint Normal Bayesian model**

$$X_t \sim \mathcal{N}(\mu, \sigma^2)$$

with conjugate priors

$$\mu \mid \sigma^2 \sim \mathcal{N}\left(\mu_0, \frac{\sigma^2}{k_0}\right)$$

$$\sigma^2 \sim \text{Inverse-Gamma} \left(\frac{r_0}{2}, \frac{r_0 \sigma_0^2}{2} \right)$$

Given observations $x_1, \dots, x_n$ with sample mean $\bar{x}$ and sample variance s^2 , the posterior updates are

$$\begin{aligned} k_n &= k_0 + n, & \mu_n &= \frac{k_0}{k_n} \mu_0 + \frac{n}{k_n} \bar{x}, \\ r_n &= r_0 + n, & \sigma_n^2 &= \frac{1}{r_n} \left(r_0 \sigma_0^2 + (n-1)s^2 + \frac{k_0}{k_n} (\bar{x} - \mu_0)^2 \right). \end{aligned}$$

These posterior quantities form the basis for the inference in this report.

Bayes Factor Calculation For Simple Hypothesis

We now compute the **Bayes Factor** to test the hypothesis

$$H_0 : \mu = 14, \text{ Vs } H_1 : \mu \neq 14$$

We perform this specific analysis because as per our results of Report 1, $\mu_0 = 14$ lead to a good approximation of the actual data.

The Bayes factor is defined as

$$BF_{01} = \frac{p(x | H_0)}{p(x | H_1)}$$

We can compute the terms $p(x | H_0)$ and $p(x | H_1)$ to get the formulas below:

$$\begin{aligned} p(x | H_1) &= \frac{\Gamma(r_n/2)}{\Gamma(r_0/2)} \sqrt{\frac{k_0}{k_n}} \frac{(r_0 \sigma_0^2/2)^{r_0/2}}{(\pi)^{n/2} (r_n \sigma_n^2/2)^{r_n/2}} \\ p(x | H_0) &= \frac{\Gamma(r'_0/2)}{\Gamma(r_0/2)} \frac{(r_0 \sigma_0^2/2)^{r_0/2}}{(\pi)^{n/2} (r'_0 \sigma_0'^2/2)^{r'_0/2}} \end{aligned}$$

where

$$r'_0 = r_0 + n, \quad \sigma_0'^2 = \frac{1}{r'_0} \left(r_0 \sigma_0^2 + \sum (x_i - 14)^2 \right)$$

Bayes Factor expression

$$BF_{01} = \sqrt{\frac{k_n}{k_0}} \frac{\Gamma(r'_0/2)}{\Gamma(r_n/2)} \frac{(r_n \sigma_n^2/2)^{r_n/2}}{(r'_0 \sigma_0'^2/2)^{r'_0/2}}$$

with

$$\begin{aligned} r'_0 &= r_0 + n, \\ \sigma_0'^2 &= \frac{1}{r'_0} \left(r_0 \sigma_0^2 + \sum_{i=1}^n (x_i - 14)^2 \right), \\ r_n &= r_0 + n, \\ \sigma_n^2 &= \frac{1}{r_n} \left(r_0 \sigma_0^2 + (n-1)s^2 + \frac{k_0 n}{k_0 + n} (\bar{x} - \mu_0)^2 \right). \end{aligned}$$

Therefore,

$$BF_{01} = \sqrt{\frac{k_n}{k_0}} \left(\frac{r_0 \sigma_0^2 + (n-1)s^2 + \frac{k_0 n}{k_0 + n} (\bar{x} - \mu_0)^2}{r_0 \sigma_0^2 + \sum_{i=1}^n (x_i - 14)^2} \right)^{r_n/2}.$$

Therefore, in order to calculate the Bayes Factor, we just need to compute the terms in the expression given above. The below code computes the Bayes Factor for different values of the mean of the prior, specifically $\mu_0 \in \{1, 14, 30\}$, each at $lag \in \{2, 3, 5, 10, 50, 100, 300\}$.

Note that the concept of lag was introduced in Project 1, as the autocorrelation factor (which measures the dependence of data at time t against data at time $t - lag$) grew with lower values of lag . By computing the Bayes Factor at different lag , we see the dependence of our Hypothesis on trends within the data itself.

```

class SimpleBayesFactor:
    def __init__(self, mu, k, sigma2, r, lag, df: pd.DataFrame):
        if min(sigma2, k, r) <= 0:
            raise ValueError("sigma2, k, r must be > 0")
        if lag <= 0:
            raise ValueError("lag must be a Natural number")

        self.lag = (lag)
        self.df = df

        self.params = {"mu": [mu], "k": [k], "sigma2": [sigma2], "r": [r]}
        self.bf01 = []

    def generate_posterior(self):
        df = self.df
        df = df.sort_values(by="timestamp").reset_index(drop=True)
        array = np.log(df["cost"].tolist())

        for i in range(len(array)):
            n = 0
            # sum_volume = 0
            # sum2_volume = 0

            window = array[max(0, i - self.lag):i]

            if len(window) < 2:
                continue

            n = len(window)

            # sample mean and sample variance calculation
            ybar = np.mean(window)
            s2 = np.var(window, ddof=1)

            # prior parameters
            mu0, k0, sigma20, r0 = self.params["mu"][0], self.params["k"][0], self.params["sigma2"][0],
self.params["r"][0]

            # posterior updates
            kn = k0 + n
            rn = r0 + n

            mun = (k0 * mu0 + n * ybar) / kn
            sigma2n = (r0 * sigma20 + (n-1) * s2 + k0 * n * ((ybar - mu0)**2) / kn) / rn

            numerator = r0 * sigma20 + (n-1) * s2 + (k0 * n * ((ybar - mu0)**2)) / kn
            denominator = r0 * sigma20 + np.pow((np.array(window) - 14), 2).sum()

            bf = np.sqrt(kn/k0) * np.pow(numerator/denominator, rn/2)

            self.params["mu"].append(mun)
            self.params["k"].append(kn)
            self.params["sigma2"].append(sigma2n)
            self.params["r"].append(rn)
            self.bf01.append(bf)

        mu0_values = [1, 14, 30]
        lags = [300, 100, 50, 10, 5, 3, 2]

    for i, lag in enumerate(lags):
        fig, axs = plt.subplots(1, len(mu0_values), figsize=(16, 4), sharex=True, sharey=True)
        for j, mu0 in enumerate(mu0_values):
            ax = axs[j]

```

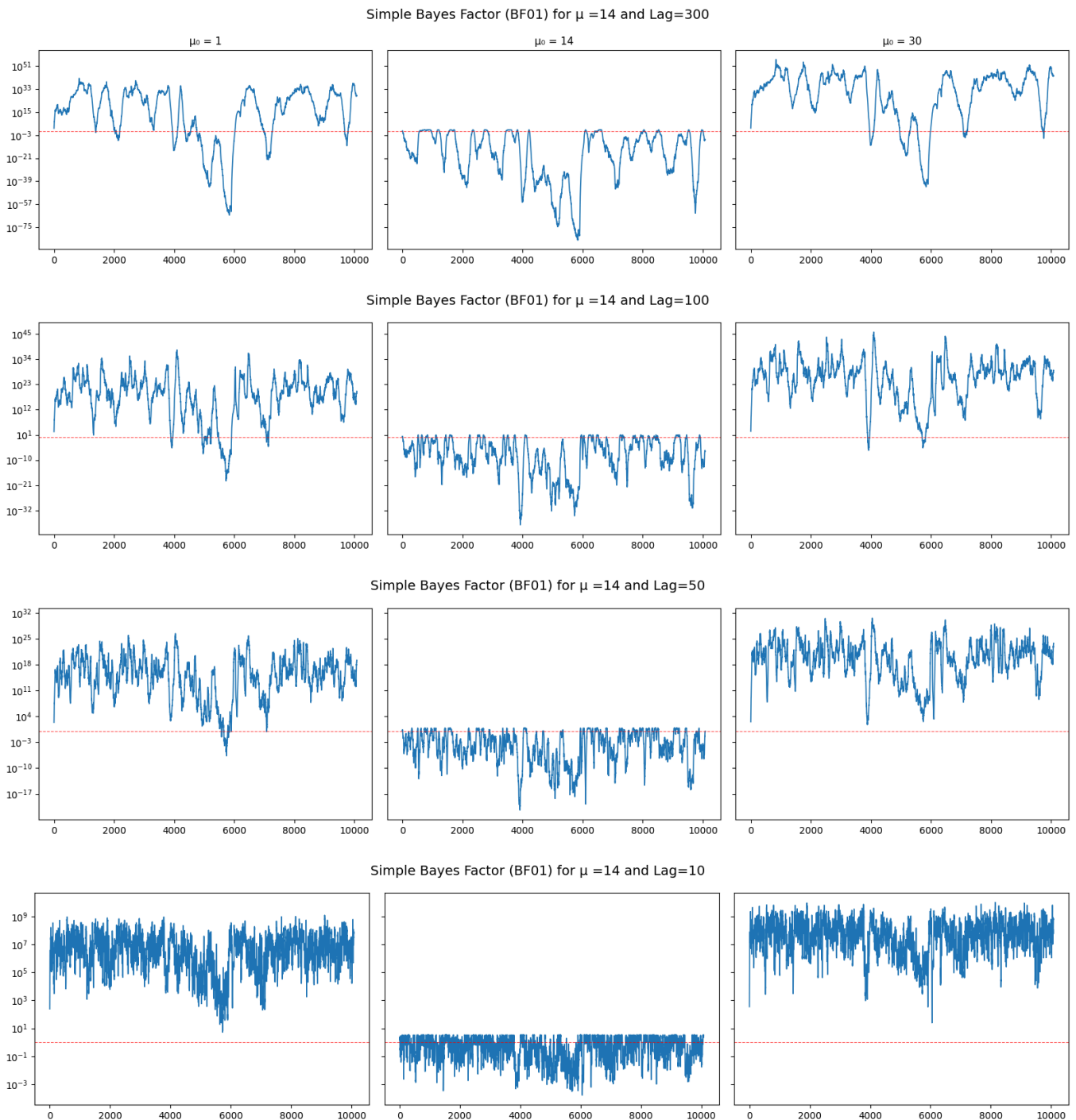
```

bf = SimpleBayesFactor(mu=mu0, k=1, sigma2=4, r=1, lag=lag, df=df)
bf.generate_posterior()

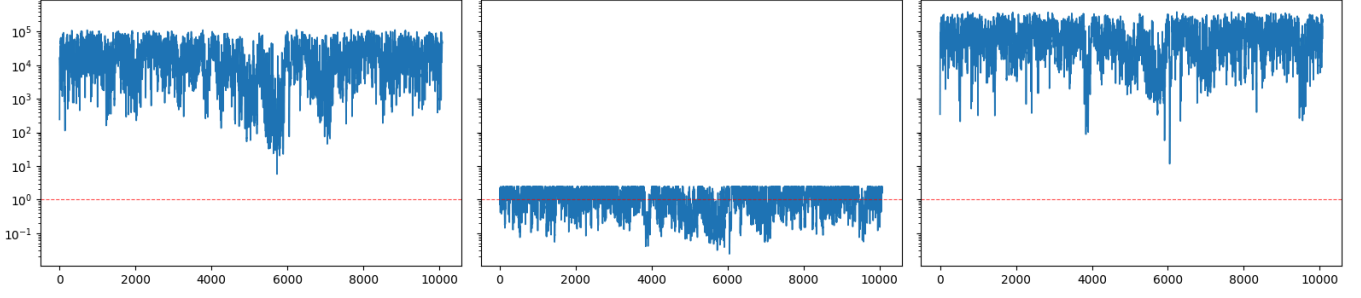
ax.plot(bf.bf01, linewidth=1.2)
ax.axhline(1, color="red", linestyle="--", linewidth=0.8, alpha=0.7)
if i == 0:
    ax.set_title(f" $\mu_0 = \{\mu_0\}$ ", fontsize=11)
# if j == 0:
#     ax.set_ylabel(f"Lag = {lag}", fontsize=10, rotation=0, labelpad=40)
# ax.grid(alpha=0.3)
ax.set_yscale("log")
plt.suptitle(f"Simple Bayes Factor (BF01) for  $\mu = 14$  and Lag={lag}", fontsize=14)
plt.tight_layout()
filename = f"Bayes_Factor_SLag_{lag}.png"
file_path = os.path.join("../figures/", filename)
plt.savefig(file_path, format='png')
plt.show()

```

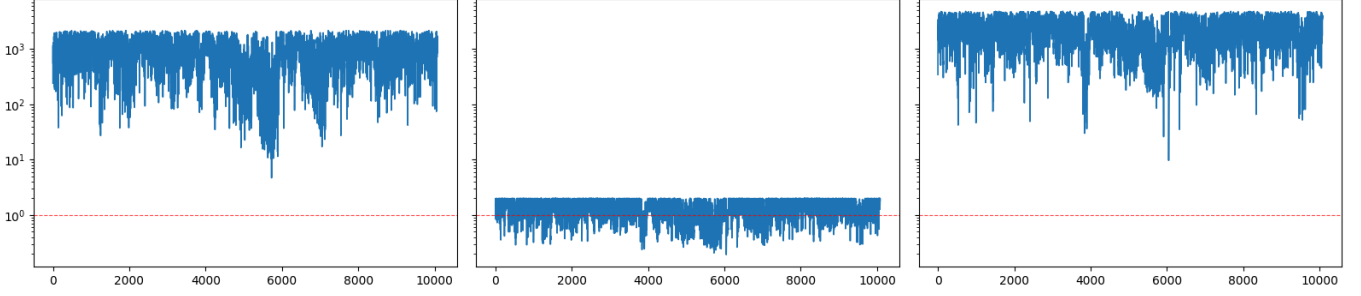
Using this code, the following plots were obtained:



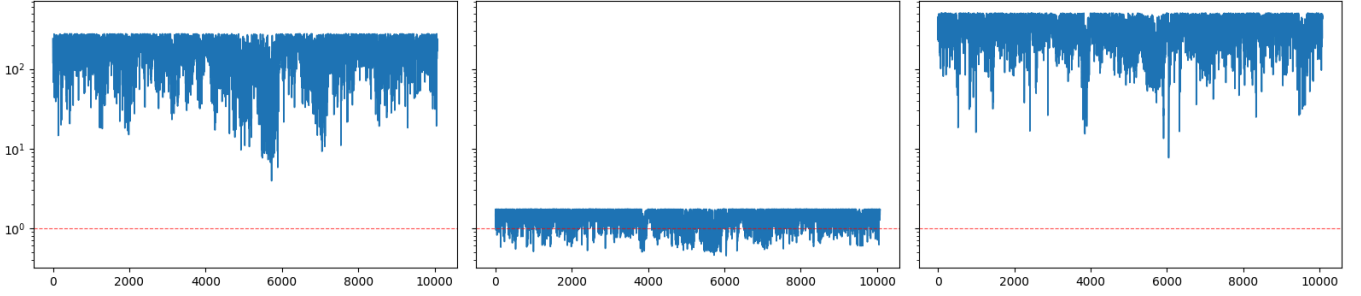
Simple Bayes Factor (BF01) for $\mu = 14$ and Lag=5



Simple Bayes Factor (BF01) for $\mu = 14$ and Lag=3



Simple Bayes Factor (BF01) for $\mu = 14$ and Lag=2



From the above Bayes factor plots, we observe that when the prior mean under the alternative is set to $\mu_0 = 14$, the Bayes factor BF_{01} is less than 1 for most of the simulations. This indicates that the composite alternative H_1 tends to explain the data better than the simple null hypothesis $H_0 : \mu = 14$. This behavior is expected because, even though both hypotheses are centered at the same value, the alternative allows μ to move away from 14. This additional flexibility often increases the marginal likelihood $p(x|H_1)$ relative to the fixed point likelihood $p(x|H_0)$, producing $BF_{01} < 1$. In other words, integrating the likelihood over a range of possible μ values can outperform a single fixed value, even when that fixed value matches the prior center. (This corresponds to point A.)

However, when the prior mean under the alternative is moved far from the data, such as $\mu_0 = 1$ or $\mu_0 = 30$, the Bayes factor becomes very large, strongly favoring the null hypothesis. In these cases, the alternative prior places most of its mass in regions that do not match the observed data, which sharply reduces the marginal likelihood under H_1 . This causes BF_{01} to increase dramatically in favor of H_0 .

Overall, combining these patterns, the plots suggest that the data is concentrated near $\mu_0 = 14$. When the alternative prior is centered near 14, it can outperform the point null simply because it allows small movements around that value; but when the prior is shifted far away, the alternative becomes incompatible with the data and is strongly rejected. This supports the conclusion that the true mean μ lies close to 14, providing a natural motivation to proceed to the composite-hypothesis comparison with μ restricted to a neighborhood around 14.

Bayes Factor Calculation for Composite Hypothesis

Now, let us compute the Bayes Factor for the composite hypothesis

$$H_0 : \mu \in (13, 15) \text{ Vs } H_1 : \mu \notin (13, 15)$$

The model is the joint Normal-Inverse-Gamma distribution, and the Bayes Factor is defined as the ratio of the posterior odds to the prior odds:

$$BF_{01} = \frac{P(H_0|D)/(1 - P(H_0|D))}{P(H_0)/(1 - P(H_0))}$$

We have first computed the marginal distribution before and after observing the data D , which we will be using to compute the probabilities of hypothesis before and after observing the data. The compute marginal distributions for μ and $\mu|D$ are

$$p(\mu) = \frac{\Gamma(\frac{r_0+1}{2})}{\Gamma(\frac{r_0}{2})\sqrt{\pi r_0(\frac{\sigma_0^2}{k_0})}} \left(1 + \frac{(\mu - \mu_0)^2}{r_0(\frac{\sigma_0^2}{k_0})}\right)^{-\frac{r_0+1}{2}}$$

and

$$p(\mu|D) = \frac{\Gamma(\frac{r_n+1}{2})}{\Gamma(\frac{r_n}{2})\sqrt{\pi r_n(\frac{\sigma_n^2}{k_n})}} \left(1 + \frac{(\mu - \mu_n)^2}{r_n(\frac{\sigma_n^2}{k_n})}\right)^{-\frac{r_n+1}{2}}.$$

By comparing our derived PDF with the canonical PDF of a non-standardized Student's t-distribution, which is defined by a degrees of freedom parameter ν , a location parameter μ_{loc} , and a scale parameter σ_{scale} :

$$p(x) = \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})\sqrt{\pi\nu\sigma_{scale}^2}} \left(1 + \frac{(x - \mu_{loc})^2}{\nu\sigma_{scale}^2}\right)^{-\frac{\nu+1}{2}}$$

we see that both the prior distribution of μ and the posterior distribution $\mu | D$ are Student's t-distributions. Specifically,

- $\mu \sim t_{r_0}(\mu_0, \sqrt{\sigma_0^2/k_0})$
- $\mu | D \sim t_{r_n}(\mu_n, \sqrt{\sigma_n^2/k_n})$

with $r_0, \mu_0, \sigma_0^2, k_0$ denoting the prior degrees of freedom, location, scale, and precision-weight parameters, and $r_n, \mu_n, \sigma_n^2, k_n$ representing their posterior counterparts.

After mathematically deriving the marginal distributions, we now compute the required probabilities using their Cumulative Distribution Functions (CDFs). Let $F_{t(\nu, \mu_{loc}, \sigma_{scale})}$ denote the CDF of the specified Student's t-distribution.

Prior Probability $P(H_0)$:

$$P(H_0) = P(\mu \in (a, b)) = \int_a^b p(\mu) d\mu = F_{t(r_0, \mu_0, \sqrt{\sigma_0^2/k_0})}(b) - F_{t(r_0, \mu_0, \sqrt{\sigma_0^2/k_0})}(a)$$

Posterior Probability $P(H_0|D)$:

$$P(H_0|D) = P(\mu \in (a, b)|D) = \int_a^b p(\mu|D) d\mu = F_{t(r_n, \mu_n, \sqrt{\sigma_n^2/k_n})}(b) - F_{t(r_n, \mu_n, \sqrt{\sigma_n^2/k_n})}(a)$$

Final Bayes Factor:

The Bayes Factor BF_{01} is assembled by substituting these probabilities into the odds ratio formula. Let $\pi_0 = P(H_0)$ and $\pi_{0|D} = P(H_0|D)$.

$$BF_{01} = \frac{\pi_{0|D}/(1 - \pi_{0|D})}{\pi_0/(1 - \pi_0)}$$

Substituting the CDF expressions:

$$BF_{01} = \frac{\frac{F_{post}(b) - F_{post}(a)}{1 - (F_{post}(b) - F_{post}(a))}}{\frac{F_{prior}(b) - F_{prior}(a)}{1 - (F_{prior}(b) - F_{prior}(a))}}$$

where F_{prior} and F_{post} denote the CDFs of the prior and posterior Student's t-distributions derived above, and we evaluate the evidence over the interval $[a, b] = [13, 15]$.

The following code computes the Bayes factor for different choices of the prior mean, specifically $\mu_0 \in \{1, 14, 30\}$, and for a range of lag values $\{2, 3, 5, 10, 50, 100, 300\}$.

```
class CompositeBayesFactor:
    def __init__(self, mu: float, k: float, sigma2: float, r: float,
                 a: float, b: float, lag: int, df: pd.DataFrame):
        if min(sigma2, k, r) <= 0:
            raise ValueError("sigma2, k, r must be > 0")
        if lag <= 0:
            raise ValueError("lag must be a Natural number")
        if a >= b:
            raise ValueError("The lower bound 'a' must be less than the upper bound 'b'")
```

```

self.lag = lag
self.df = df
self.interval = (a, b)
self.prior_params = {"mu0": mu, "k0": k, "sigma20": sigma2, "r0": r}
self.posterior_history = []
self.bf01 = []

def generate_posterior(self):
    df_sorted = self.df.sort_values(by="timestamp").reset_index(drop=True)
    log_cost_array = np.log(df_sorted["cost"].to_numpy())

    for i in range(len(log_cost_array)):
        window = log_cost_array[max(0, i - self.lag) : i]
        if len(window) < 2:
            continue

        n = len(window)
        ybar = np.mean(window)
        s2 = np.var(window, ddof=1) if n > 1 else 0

        mu0 = self.prior_params["mu0"]
        k0 = self.prior_params["k0"]
        sigma20 = self.prior_params["sigma20"]
        r0 = self.prior_params["r0"]

        # Standard posterior update rules
        kn = k0 + n
        rn = r0 + n
        mun = (k0 * mu0 + n * ybar) / kn
        sigma2n = (r0 * sigma20 + (n - 1) * s2 + (k0 * n * (ybar - mu0)**2) / kn) / rn

        current_posterior = {"mun": mun, "kn": kn, "sigma2n": sigma2n, "rn": rn}
        self.posterior_history.append(current_posterior)

        a, b = self.interval

        # --- Prior Probability P(H0) ---
        prior_dof = r0
        prior_loc = mu0
        prior_scale = np.sqrt(sigma20 / k0)

        p_h0 = t.cdf(b, df=prior_dof, loc=prior_loc, scale=prior_scale) - t.cdf(a, df=prior_dof,
loc=prior_loc, scale=prior_scale)
        log_prior_odds = np.log(p_h0) - np.log1p(-p_h0)

        # --- Posterior Probability P(H0|D) ---
        post_dof = rn
        post_loc = mun
        post_scale = np.sqrt(sigma2n / kn)

        p_h0_given_d = t.cdf(b, df=post_dof, loc=post_loc, scale=post_scale) - t.cdf(a, df=post_dof,
loc=post_loc, scale=post_scale)

        # Calculating the Bayes Factor
        # BF01 = Posterior Odds / Prior Odds
        if p_h0_given_d == 1.0:
            # If p=1, log-odds are infinite
            log_posterior_odds = np.inf
        elif p_h0_given_d == 0.0:
            # If p=0, log-odds are negative infinite
            log_posterior_odds = -np.inf
        else:
            log_posterior_odds = np.log(p_h0_given_d) - np.log1p(-p_h0_given_d)

        log_bf = log_posterior_odds - log_prior_odds

```

```

        self.bf01.append(log_bf)

    mu0_values = [1, 14, 30]
lags = [300, 100, 50, 10, 5, 3, 2]
a= 13
b= 15

for i, lag in enumerate(lags):
    fig, axs = plt.subplots(1, len(mu0_values), figsize=(16, 4), sharex=True, sharey=True)
    for j, mu0 in enumerate(mu0_values):
        ax = axs[j]

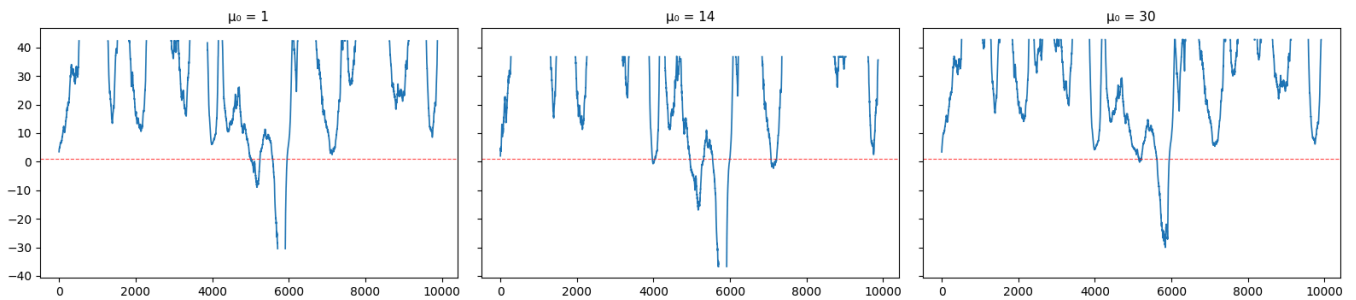
        bf = CompositeBayesFactor(
            mu=mu0,
            k=1.0,
            sigma2=1.0,
            r=1.0,
            a=a,
            b=b,
            lag=lag,
            df=df
        )
        bf.generate_posterior()

        ax.plot(bf.bf01, linewidth=1.2)
        ax.axhline(1, color="red", linestyle="--", linewidth=0.8, alpha=0.7)
        if i == 0:
            ax.set_title(f" $\mu_0 = \{mu0\}$ ", fontsize=11)
        # if j == 0:
        #     ax.set_ylabel("Lag = {lag}", fontsize=10, rotation=0, labelpad=40)
        # ax.grid(alpha=0.3)
        # ax.set_yscale("log")
    plt.suptitle(f"Composite Bayes Factor (BF01) for  $\mu \in \{a\}, \{b\}$  and Lag={lag}", fontsize=14)
    plt.tight_layout()
    filename = f"Bayes_Factor_CLag_{lag}.png"
    file_path = os.path.join("../figures/", filename)
    plt.savefig(file_path, format='png')
    plt.show()

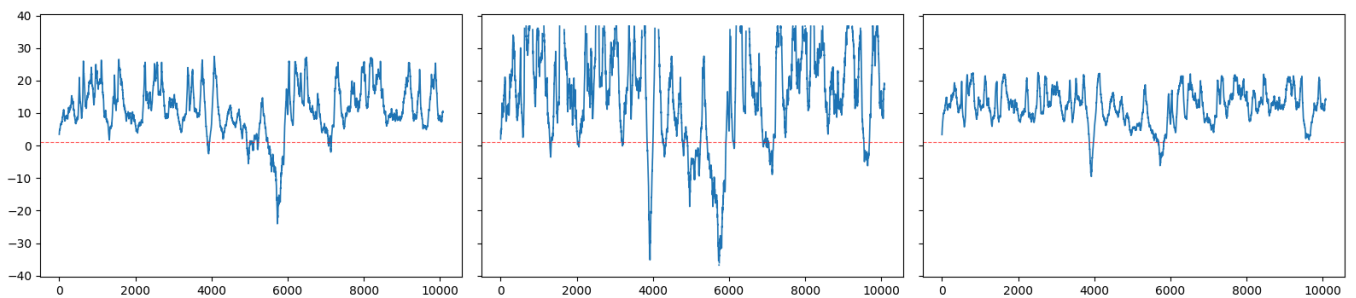
```

Using the above code the following plots were obtained:

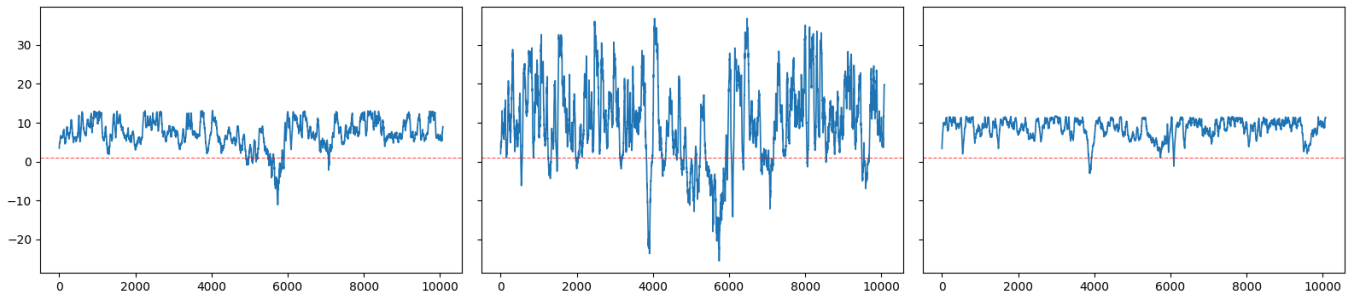
Composite Bayes Factor (BF01) for $\mu \in (13,15)$ and Lag=300



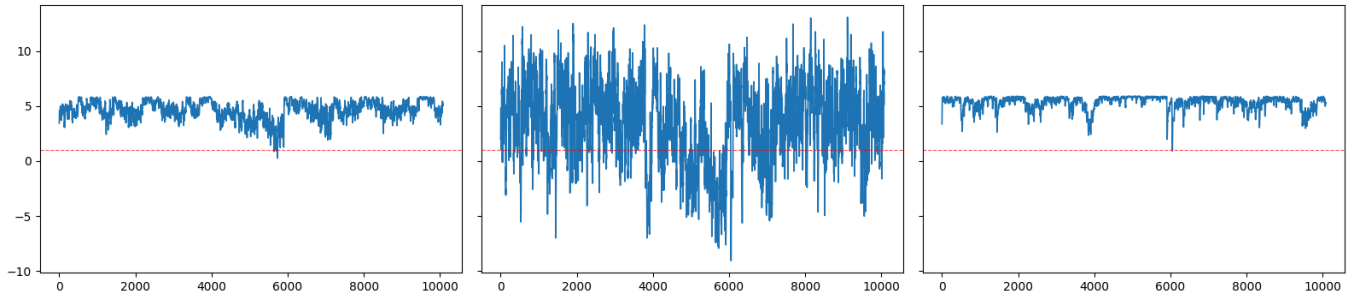
Composite Bayes Factor (BF01) for $\mu \in (13,15)$ and Lag=100



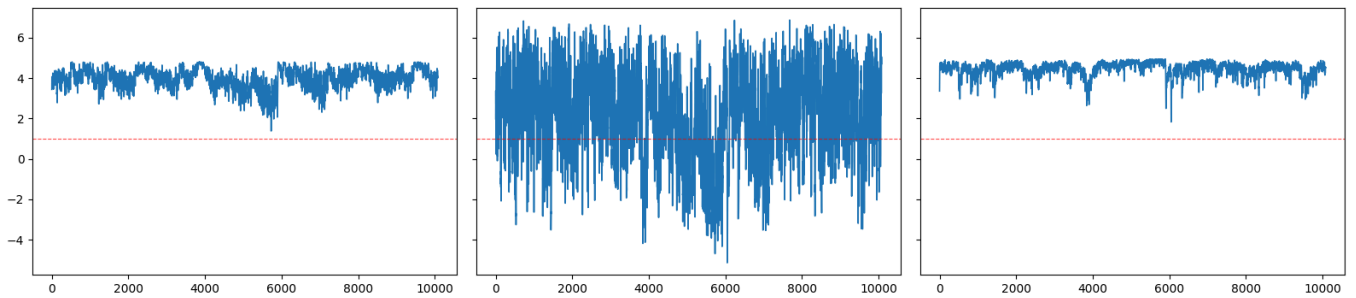
Composite Bayes Factor (BF01) for $\mu \in (13,15)$ and Lag=50



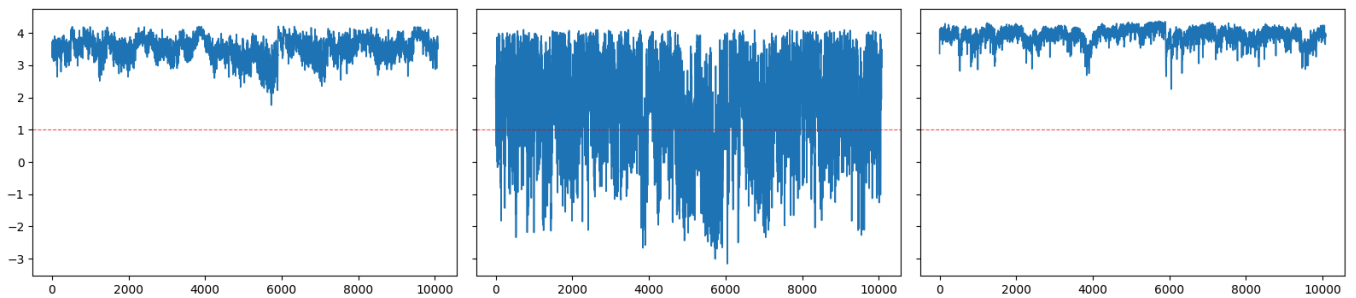
Composite Bayes Factor (BF01) for $\mu \in (13,15)$ and Lag=10



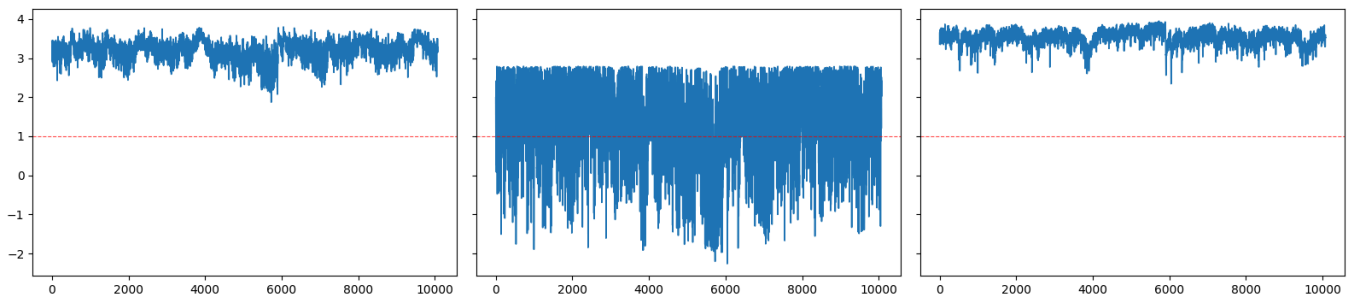
Composite Bayes Factor (BF01) for $\mu \in (13,15)$ and Lag=5



Composite Bayes Factor (BF01) for $\mu \in (13,15)$ and Lag=3



Composite Bayes Factor (BF01) for $\mu \in (13,15)$ and Lag=2



The above plots show how the Bayes factor for the composite hypothesis ($\mu \in (13,15)$) behaves as the lag (window size) decreases. For large windows (lags 300, 100, 50), the log Bayes factor stays strongly positive and stable, giving clear evidence that the mean lies within the interval. The large sample size smooths out noise, producing reliable and consistent inference.

At intermediate windows (lag 10), the Bayes factor remains mostly positive but becomes noticeably more variable. The evidence still favors the hypothesis, but short-term fluctuations begin to influence the result.

For very small windows (lags 5, 3, 2), the Bayes factor becomes highly unstable, switching frequently between supporting and rejecting the hypothesis. With so little data, noise overwhelms the signal, making any inference about the true mean unreliable.

Overall, the results show that long-term windows provide strong, stable evidence that (μ) lies within (13, 15), while short-term windows lead to erratic and untrustworthy conclusions. This highlights the importance of choosing an appropriate timescale when analyzing the process.

Sampling Algorithms

So far we have examined Bayes factor calculations under both simple and composite hypotheses, we now turn to the tools needed for posterior computation when analytical solutions are no longer available. Many of the models used in Bayesian analysis, especially those involving hierarchical structures or non-conjugate likelihoods produce posterior distributions that cannot be expressed in closed form. In such cases, we rely on **sampling algorithms** to approximate the posterior. In this sections, we introduce two fundamental approaches: **grid approximation**, which provides an intuitive, brute-force method for exploring posterior shapes, and the **Metropolis algorithm**, a Markov chain Monte Carlo (MCMC) technique that enables sampling from complex, high-dimensional distributions. Later, in the Bayesian regression section, we will extend these ideas using **Gibbs sampling**, which leverages conditional distributions for more efficient posterior exploration.

We model count data Y_t (e.g., trade counts) using a **Poisson likelihood** with a **log-normal prior** on the rate λ_t :

$$Y_t \mid \lambda_t \sim \text{Poisson}(\lambda_t)$$

$$\lambda_t \sim \text{LogNormal}(\mu, \sigma^2)$$

Posterior Distribution

The posterior of λ_t given past counts $Y_{t-\text{lag}:t-1}$ is:

$$p(\lambda_t \mid \mathbf{Y}_{t-\text{lag}:t-1}) \propto p(\mathbf{Y}_{t-\text{lag}:t-1} \mid \lambda_t) p(\lambda_t)$$

Using the Likelihood distribution and prior distribution, we obtain the following formula for the posterior distribution:

$$p(\lambda_t \mid \mathbf{Y}_{t-\text{lag}:t-1}) \propto \left(\prod_{i=t-\text{lag}}^{t-1} \frac{\lambda_t^{Y_i} e^{-\lambda_t}}{Y_i!} \right) \frac{1}{\lambda_t \sigma \sqrt{2\pi}} \exp \left(- \frac{(\log \lambda_t - \mu)^2}{2\sigma^2} \right)$$

Grid Approximation

We first perform grid approximation to sample the posterior. We assume that $\lambda \in \{0.1, 50.60505, 101.1101, 151.61515, \dots, 5000.0\}$. These are the points obtained when we assume that $\lambda_{\min} = 0.1$, $\lambda_{\max} = 5000$ and we take 100 equidistant points within this minimum and maximum.

For each of these points, we compute the unnormalized posterior probability and then normalize the probabilities using the probabilities taken at the discrete points so that they sum to 1. Finally, we draw samples from this discrete posterior distribution to approximate the behavior of λ .

The code for generating samples from this posterior distribution, and then plotting the mean of these samples against the original data is given below.

```
class LogNormalPoissonGrid:
    def __init__(self, mean, variance, lag, df: pd.DataFrame):
        self._mu = mean
        self._sigma = math.sqrt(variance)
        self.lag = lag
        self.df = df.sort_values(by="timestamp").reset_index(drop=True)
        self.counts = self.df["trade_count"].values
        self.posterior_means = []
        self.ppi = {}

    def _log_prior(self, lambda_val):
        return lognorm.logpdf(lambda_val, s=self._sigma, scale=math.exp(self._mu))

    def _log_likelihood(self, lambda_val, past_counts):
        if lambda_val <= 0:
            return -np.inf
        return np.sum(poisson.logpmf(past_counts, mu=lambda_val))

    def run_grid_inference(self, lambda_min, lambda_max, num_points=1000, ppi_level=0.95):
        grid = np.linspace(lambda_min, lambda_max, num_points)
        n = len(self.counts)
        posterior_means = []
```

```

ppi_lower = []
ppi_upper = []

prior_mean = math.exp(self._mu + (self._sigma**2) / 2)

for i in tqdm(range(n), desc="Running grid inference"):
    past_counts = self.counts[max(0, i - self.lag):i]

    if len(past_counts) == 0:
        posterior_means.append(prior_mean)
        prior_lambdas = lognorm.rvs(s=self._sigma, scale=math.exp(self._mu), size=1000)
        pred_samples = poisson.rvs(mu=prior_lambdas)
        ppi_lower.append(np.percentile(pred_samples, (1-ppi_level)/2*100))
        ppi_upper.append(np.percentile(pred_samples, (1+ppi_level)/2*100))
        continue

    log_post = np.array([self._log_likelihood(lam, past_counts) + self._log_prior(lam) for lam in
grid])

    post_unnorm = np.exp(log_post - np.max(log_post))
    post = post_unnorm / post_unnorm.sum()

    posterior_mean = np.sum(grid * post)
    posterior_means.append(posterior_mean)

    pred_samples = []
    for lam, w in zip(grid, post):
        n_samples = int(w * 1000)
        if n_samples > 0:
            pred_samples.append(poisson.rvs(mu=lam, size=n_samples))

    pred_samples = np.concatenate(pred_samples)
    ppi_lower.append(np.percentile(pred_samples, (1-ppi_level)/2*100))
    ppi_upper.append(np.percentile(pred_samples, (1+ppi_level)/2*100))

self.posterior_means = posterior_means
self.ppi[ppi_level] = {"lower_bound": ppi_lower, "upper_bound": ppi_upper}

prior_log_mean = 2.0
prior_log_variance = 0.25
lag = 10

model_grid = LogNormalPoissonGrid(
    mean=prior_log_mean,
    variance=prior_log_variance,
    lag=lag,
    df=df
)

model_grid.run_grid_inference(lambda_min=0.01, lambda_max=5000, num_points=100)

plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(16, 8))

ax.plot(df['timestamp'], df['trade_count'], 'o', color='gray', alpha=0.6, ms=4, label='Actual Trade
Count')
ax.plot(df['timestamp'], model_grid.posterior_means, color='#d95f02', lw=2.5, label='Posterior Mean Rate
( $\lambda$ )')

ppi_95 = model_grid.ppi[0.95]
ax.fill_between(df['timestamp'], ppi_95['lower_bound'], ppi_95['upper_bound'], color='#7570b3', alpha=0.3,
label='95% Predictive Interval')

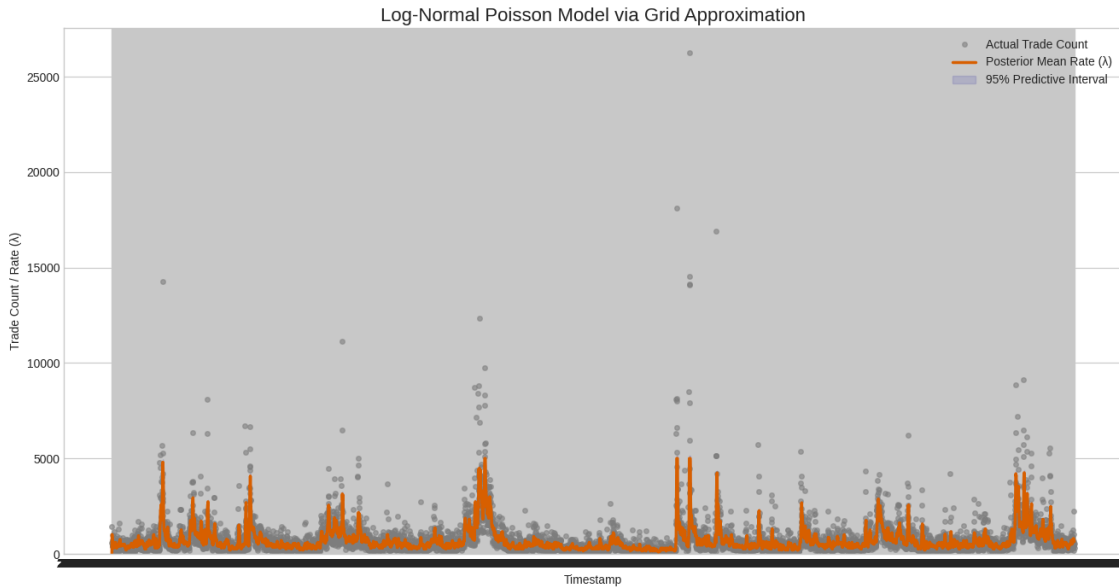
ax.set_title('Log-Normal Poisson Model via Grid Approximation', fontsize=16)
ax.set_xlabel('Timestamp')
ax.set_ylabel('Trade Count / Rate ( $\lambda$ )')
ax.legend()

```

```
ax.set_ylim(bottom=0)

filename = f"LP_lag_{lag}_Grid.png"
file_path = os.path.join("../figures/", filename)
plt.savefig(file_path, format='png')
plt.show()
```

The plot obtained from the same is below:



We observe that this approximation fails to capture the high values of λ on some days. This was bound to happen because we chose our max value of λ to be 5000. Also, since we chose only 100 points in our interval, our approximation is not smooth. We would have a much finer approximation if we chose to have more points, however this would lead to higher computational complexity, which leads to infeasible run times. Note that a low value of λ_{max} is chosen to keep the posterior prediction smooth despite the low number of points.

But we still observe that this approximation is a decent estimate of the actual data, and simulates the trends of rise and falls fairly well. A lag value of 10 was used to generate the posterior. The high autocorrelation likely leads to a better approximation in this case.

Metropolis Hastings Algorithm

We now try another method for sampling from the posterior. This method is known as the Metropolis Hastings Algorithm, where we generate a Markov Chain of samples whose stationary distribution goes to the posterior.

The proposal distribution used in our Metropolis Hastings Algorithm is the Log Normal Distribution. In particular, we fix a proposal width and then generate $\lambda_{proposed}$ from the following distribution:

$$\log(\lambda_{proposed}) | \lambda_{current} \sim \mathcal{N}(\log(\lambda_{current}), \text{proposal_width}^2)$$

The proposal width chosen for our analysis is 0.5. Note that $\log(\lambda)$ follows a symmetric distribution. Therefore, this can be said to be a Metropolis Algorithm.

Once we have the proposal distribution, we compute the acceptance probability of the new $\lambda_{proposed}$ as:

$$\alpha = \min \left(1, \frac{p(\lambda_{proposed} | \text{data})}{p(\lambda_{current} | \text{data})} \right)$$

If $\alpha = 1$, then $\lambda_{new} = \lambda_{proposed}$, otherwise

1. Generate u from $\text{Unif}(0, 1)$ distribution.
2. $\lambda_{new} = \lambda_{proposed}$ if $u \leq \alpha$.
3. $\lambda_{new} = \lambda_{current}$ if $u > \alpha$

We also plot the same posterior mean against the original data to see how well the MH algorithm performs.

The code for MH Algorithm is given below:

```
class LogNormalPoissonMH:
    def __init__(self, mean, variance, lag, df: pd.DataFrame):
        if variance < 0:
            raise ValueError("Variance must be positive")
        if lag <= 0:
            raise ValueError("Lag must be positive")

        # Parameters for the Log-Normal prior on lambda
        self._mu = mean
        self._sigma = math.sqrt(variance)

        self.lag = lag
        self.df = df.sort_values(by="timestamp").reset_index(drop=True)
        self.counts = self.df["trade_count"].values

        # --- Results ---
        self.posterior_means = []
        self.ppi = {}

    def _log_prior(self, lambda_val):
        return lognorm.logpdf(lambda_val, s=self._sigma, scale=math.exp(self._mu))

    def _log_likelihood(self, lambda_val, past_counts):
        if lambda_val <= 0:
            return -np.inf
        return np.sum(poisson.logpmf(past_counts, mu=lambda_val))

    def _mcmc_for_one_step(self, past_counts, n_samples, burn_in, proposal_width):
        lambda_current = math.exp(self._mu + (self._sigma**2) / 2)

        samples = []

        for i in range(n_samples + burn_in):
            log_lambda_proposal = np.random.normal(loc=math.log(lambda_current), scale=proposal_width)
            lambda_proposal = math.exp(log_lambda_proposal)

            log_post_current = self._log_likelihood(lambda_current, past_counts) +
            self._log_prior(lambda_current)
            log_post_proposal = self._log_likelihood(lambda_proposal, past_counts) +
            self._log_prior(lambda_proposal)

            log_acceptance_ratio = log_post_proposal - log_post_current

            if math.log(np.random.rand()) < log_acceptance_ratio:
                lambda_current = lambda_proposal

            if i >= burn_in:
                samples.append(lambda_current)

        return np.array(samples)

    def run_mcmc_inference(self, n_mcmc_samples=1000, burn_in=200, proposal_width=0.5):
        n = len(self.counts)
        prior_mean = math.exp(self._mu + (self._sigma**2) / 2)

        posterior_means = []
        ppi_lower = []
        ppi_upper = []

        print("Running MCMC inference for each time step...")
        for i in tqdm(range(n)):
            start_idx = max(0, i - self.lag)
            past_counts = self.counts[start_idx:i]
```

```

        if len(past_counts) == 0:
            posterior_means.append(prior_mean)
            prior_lambdas = lognorm.rvs(s=self._sigma, scale=math.exp(self._mu), size=n_mcmc_samples)
            pred_samples = poisson.rvs(mu=prior_lambdas)
            ppi_lower.append(np.percentile(pred_samples, 2.5))
            ppi_upper.append(np.percentile(pred_samples, 97.5))
            continue

        posterior_lambda_samples = self._mcmc_for_one_step(
            past_counts, n_mcmc_samples, burn_in, proposal_width
        )

        posterior_means.append(np.mean(posterior_lambda_samples))

        predictive_samples = poisson.rvs(mu=posterior_lambda_samples)

        ppi_lower.append(np.percentile(predictive_samples, 2.5))
        ppi_upper.append(np.percentile(predictive_samples, 97.5))

    self.posterior_means = posterior_means
    self.ppi[0.95] = {"lower_bound": ppi_lower, "upper_bound": ppi_upper}
    print("Inference complete.")

prior_log_mean = 2.0
prior_log_variance = 0.25
lag = 10

model_mh = LogNormalPoissonMH(
    mean=prior_log_mean,
    variance=prior_log_variance,
    lag=lag,
    df=df
)

model_mh.run_mcmc_inference(n_mcmc_samples=1000, burn_in=200, proposal_width=0.3)

plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(16, 8))

ax.plot(df['timestamp'], df['trade_count'], 'o', color='gray', alpha=0.6, ms=4, label='Actual Trade Count')

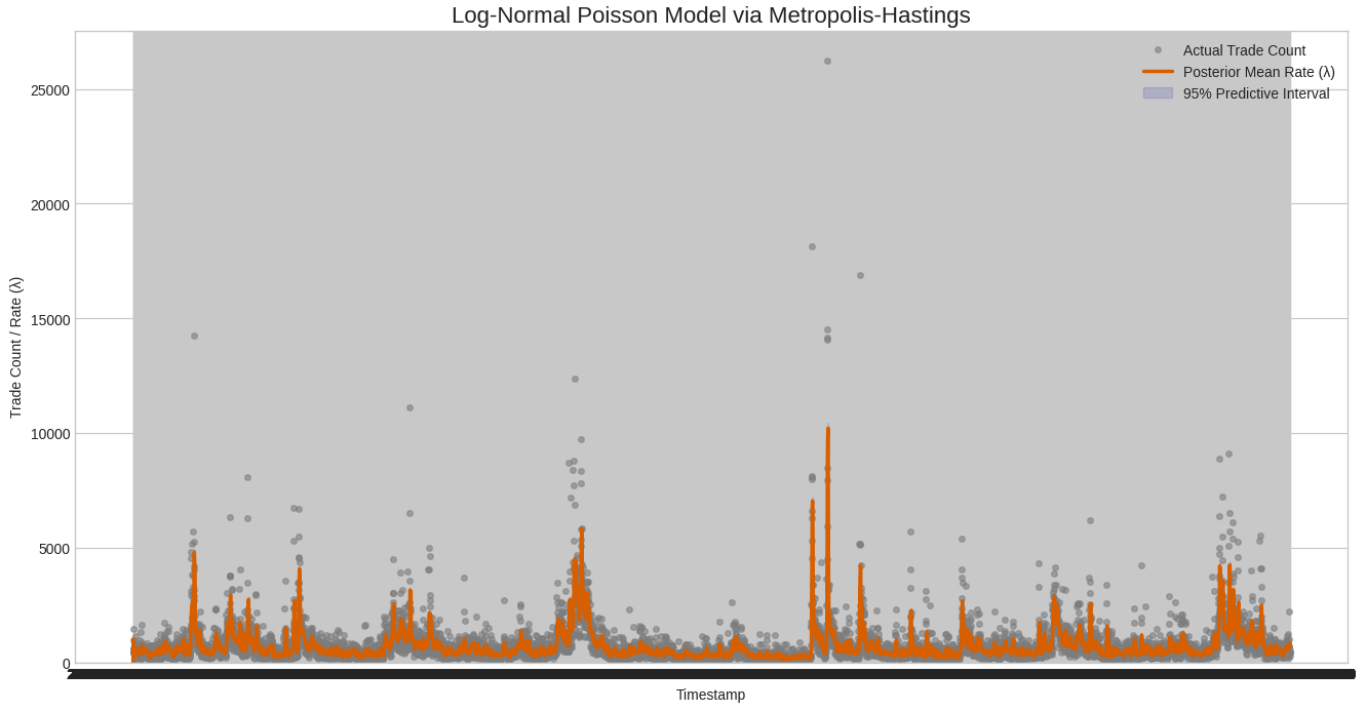
ax.plot(df['timestamp'], model_mh.posterior_means, color='#d95f02', lw=2.5, label='Posterior Mean Rate ( $\lambda$ )')

# Plot 95% Poisson Predictive Interval (PPI)
ppi_95 = model_mh.ppi[0.95]
ax.fill_between(df['timestamp'], ppi_95['lower_bound'], ppi_95['upper_bound'], color='#7570b3', alpha=0.3,
label='95% Predictive Interval')

ax.set_title('Log-Normal Poisson Model via Metropolis-Hastings', fontsize=16)
ax.set_xlabel('Timestamp')
ax.set_ylabel('Trade Count / Rate ( $\lambda$ )')
ax.legend()
ax.set_ylim(bottom=0)
filename = f"LP_lag_{lag}_MH.png"
file_path = os.path.join("../figures/", filename)
plt.savefig(file_path, format='png')
plt.show()

```

The plot obtained from this code is given below:



Firstly, we note that this sampling algorithm is able to sample large spikes more effectively as compared to Grid Approximation, that too at a lower computational cost. The shape of the Posterior Means follows the shape of the actual data.

For this analysis, we use a lag of 10, consistent with the grid approximation setup. Overall, the Metropolis-Hastings algorithm provides an effective and computationally efficient approach to posterior sampling.

Bayesian Regression for Log Returns

Returns represent the change in an asset's price from one period to the next. Predicting this change in asset price is a crucial part of many financial trading strategies. As such, we now move our focus to trying to predict the returns over every minute. Like before, the asset we will be analyzing is BTC/USDT.

However, we first note that returns can be very unpredictable and show sharp rises and falls. As such, we use the log transform in order to normalize them. Hence, we try to predict Log Returns. In particular, we predict

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

where r_t denotes the return at minute t , and P_t denotes the price of the asset at time t .

We aim to perform this prediction using a Bayesian Multiple Regression Model. The predictor variables we will use in this Bayesian Regression are listed below:

- Opening Price at time t : This is the price of the asset at the start of minute t .
- Previous Closing Price at time t : This is the price of the asset at the end of minute $t - 1$.
- log Volume: This is the logarithm of the volume of trades in minute t .
- Buy Sell Imbalance: $\frac{\text{BuyVolume}_t - \text{SellVolume}_t}{\text{BuyVolume}_t + \text{SellVolume}_t}$. It is the normalized ratio of difference in between Total Volume bought and Total Volume sold.
- Returns at time $t - 1$
- Volatility at time t : Standard deviation of the log returns in the last 10 minutes.

Thus, as per our Bayesian Regression setup, we have

$$r_t = \beta_{t,0} + \sum_{i=1}^6 \beta_{t,i} x_{t,i} = x_t^T \beta$$

where $\beta = [\beta_{t,0}, \beta_{t,1}, \dots, \beta_{t,6}]^T$ and $x_t = [1, x_{t,1}, x_{t,2}, \dots, x_{t,6}]^T$.

We further impose the following assumptions on the parameters β

$$\beta \sim \mathcal{N}(\mathbf{0}, 10\sigma^2 I), \text{ and } \sigma^2 \sim \text{IG}(2, 0.0001)$$

where IG means Inverse Gamma and I is the 7×7 identity matrix.

We now use a rolling window of times to update our prior and get a posterior distribution for β and σ^2 . In our implementation, we choose $p = 100$, so that we use the data of the last 100 samples to update our prior.

Posterior Update Rules for Bayesian Regression

Suppose we have

$$\beta \sim \mathcal{N}(\mu_0, \Sigma_0) \text{ and } \sigma^2 \sim \text{IG}(\alpha_0, \beta_0)$$

Given a lag window of data x and corresponding log returns r , the posterior of the regression parameters is updated as follows.

$$\begin{aligned}\Sigma_n &= (\Sigma_0^{-1} + X^T X)^{-1} \\ \mu_n &= \Sigma_n (\Sigma_0^{-1} \mu_0 + X^T r) \\ \alpha_n &= \alpha_0 + \frac{n}{2}, \quad \beta_n = \beta_0 + \frac{1}{2} [r^T r + \mu_0^T \Sigma_0^{-1} \mu_0 - \mu_n^T \Sigma_n^{-1} \mu_n]\end{aligned}$$

Posterior distributions:

$$\beta \mid \sigma^2, X, r \sim \mathcal{N}(\mu_n, \sigma^2 \Sigma_n), \quad \sigma^2 \mid X, r \sim \text{IG}(\alpha_n, \beta_n)$$

Implementation - Gibbs Sampling

We cannot directly sample β, σ^2 from their joint distribution, as the distribution of the Joint is not a standard distribution. However, we have the full-conditional posterior distributions for β and σ^2 . Thus, we can use **Gibbs Sampling** to generate samples from the joint posterior distribution.

We construct a Markov Chain $(\theta_0, \theta_1, \dots, \theta_n)$, where each $\theta_i = (\beta_i, \sigma_i^2)$. Once we have θ_{i-1} , generate θ_i as follows:

1. Sample $\beta^{(i)}$ from its full conditional distribution:

$$\beta^{(i)} \mid \sigma^{2(i-1)}, X, r \sim \mathcal{N}(\mu_n, \sigma^{2(i-1)} \Sigma_n)$$

where μ_n and Σ_n are computed as in the posterior update formulas.

2. Sample $\sigma^{2(i)}$ from its full conditional distribution:

$$\sigma^{2(i)} \mid \beta^{(i)}, X, r \sim \text{Inverse-Gamma}(\alpha_n, \beta_n)$$

with α_n and β_n computed using $\beta^{(i)}$.

After a large number of iterations, the sample $\{\theta_i\}_{i=1 \dots n}$ converges to the joint posterior distribution of β and σ^2 .

Using the sample of β_i , we compute the mean of the parameters, and use it to

- Predict the log returns using the mean of the parameters
- Construct 95% Credible Intervals for the log returns

In our implementation, we have chosen $\theta_0 = (\mathbf{0}, 1)$. We perform 1000 iterations and discard the first 100 iterations as the burn-in period. The code for the same is given below:

```
# Posterior of parameters given variance, data follows Normal(posterior_mean, posterior_cov * variance)

# Posterior of variance given parameters, data follows InverseGamma(posterior_alpha, posterior_beta)

def Gibbs_Sampler(prior_mean, prior_cov, x, y, alpha_prior, beta_prior, iterations = 1000, burn_in=100):
    prior_inverse = np.linalg.inv(prior_cov)
    xtx = np.transpose(x) @ x
    posterior_cov = np.linalg.inv(prior_inverse + xtx)
    posterior_mean = posterior_cov @ (prior_inverse @ prior_mean + np.transpose(x) @ y)

    posterior_alpha = alpha_prior + len(y) / 2
    d = x.shape[1]
    sample_params = np.zeros((iterations + 1, d))
    sample_variances = np.zeros(iterations + 1)
    sample_params[0] = (prior_mean)
    sample_variances[0] = (1)

    for i in range(iterations):
```



```

last_variance = sample_variances[i]

param = np.random.multivariate_normal(posterior_mean, posterior_cov * last_variance)
sample_params[i + 1] = (param)

mat = y - x @ param
posterior_beta = beta_prior + np.transpose(mat) @ mat / 2

sample_var = 1 / np.random.gamma(posterior_alpha, 1 / posterior_beta)
sample_variances[i + 1] = (sample_var)

return sample_params[burn_in:], sample_variances[burn_in:]

```

The code used to perform Bayesian Regression at time t is given below:

```

# p is lag window, will use data of last p days to train regression
# Assume beta follows Normal(prior_mean, sigma^2 prior_cov)
# Error term follows Normal(0, sigma^2)
# sigma^2 follows Inverse Gamma (alpha, beta)

def Regression(p, time, prior_mean, prior_cov, alpha, beta):
    data = df.iloc[max(0, time - p): time]
    X = data[["open_price", "prev_close_price", "log_volume", "buy_sell_imbalance", "lag1_return",
"volatility"]]
    X = data[[
        "open_price",
        "prev_close_price",
        "log_volume",
        "buy_sell_imbalance",
        "lag1_return",
        "volatility"
    ]].copy()
    X["const"] = 1

    x = X.to_numpy()
    Y = data["log_return"]
    y = Y.to_numpy()

    sample_params, sample_variances = Gibbs_Sampler(prior_mean, prior_cov, x, y, alpha, beta)

    n = len(sample_params)
    x_current = df.loc[time, ["open_price", "prev_close_price", "log_volume", "buy_sell_imbalance",
"lag1_return", "volatility"]].to_numpy()
    x_current = np.append(x_current, 1)

    y_pred_samples = []
    for i in range(n):
        error = np.random.normal(0, np.sqrt(sample_variances[i]))
        params = sample_params[i]

        y_pred = params @ x_current + error
        y_pred_samples.append(y_pred)

    mean = np.mean(y_pred_samples)
    lb = np.quantile(y_pred_samples, 0.025)
    ub = np.quantile(y_pred_samples, 0.975)

    return mean, lb, ub

def get_predictions(p):
    n = df.shape[0]
    feature_count = 7
    # Choosing priors
    # prior_mean is 0
    # prior_covariance matrix is identity matrix * 10
    prior_mean = np.zeros(feature_count)

```

```

prior_cov = np.eye(feature_count) * 10
alpha_prior = 2
beta_prior = 0.0001
means = []
lower_bounds = []
upper_bounds = []

for i in tqdm(range(n), total=n, desc="Processing log returns"):
    mean, ci_lb, ci_ub = Regression(min(p, i), i, prior_mean, prior_cov, alpha_prior, beta_prior)

    means.append(mean)
    lower_bounds.append(ci_lb)
    upper_bounds.append(ci_ub)

return means, lower_bounds, upper_bounds

np.random.seed(0)
mean, lb, ub = get_predictions(100)

```

We plotted the prediction of the log returns vs the actual log returns. The code for the same is below.

```

def plot_prediction_means(p):
    mean, lb, ub = get_predictions(100)
    y = df["log_return"].to_numpy()
    plt.figure(figsize=(12, 5))
    plt.plot(y[1:], label="Actual log returns", color="blue")
    plt.plot(mean[1:], label="Predicted posterior mean", color="red", linestyle="--")
    plt.xlabel("Time (minutes)")
    plt.ylabel("Log return")
    plt.title("BTC/USDT Log Returns vs Predicted Posterior Means")
    plt.legend()
    filename = f"regression_post_mean.png"
    file_path = os.path.join("../figures/", filename)
    plt.savefig(file_path, format='png')
    plt.show()

plot_prediction_means(100)

```

We also found the 95% confidence intervals for each time t , and found how many points actually lie within the Central Posterior Interval.

```

def count_correct_ppi(p):
    _, lb, ub = get_predictions(p)
    y = df["log_return"].to_numpy()
    count = 0
    for i in range(len(mean)):
        if y[i] >= lb[i] and y[i] <= ub[i]:
            count += 1

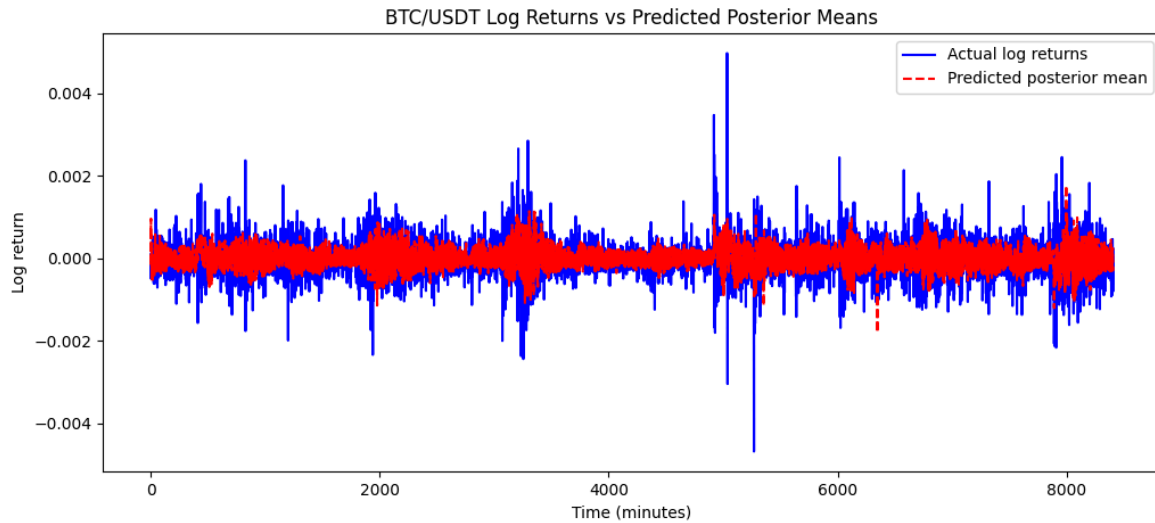
    print(100 * count / len(mean))

count_correct_ppi(100)

```

Firstly, we see that 99.96432818073721% of the actual log returns lie within the Central Posterior Interval. This shows that our predictor is very good at approximating close to the actual returns. However, this high percentage of points lying in the CPI may be due to the extremely small values of log returns. The Posterior intervals are fairly large in comparison to the actual value of log returns.

Using the above code, the following plot for Prediction vs Actual Log returns was obtained:



We observe that the shape of the posterior emulates the actual log returns fairly well. However, it is unable to match the sharp rises/drops. Nonetheless, this model performs well with regards to predictions.

Hierarchical Modelling

In Project-1, we have modeled the trade counts using Gamma–Poisson and LogNormal–Poisson formulations. While both models were capable of capturing the overall trend, they struggled to represent the full variability in the data most notably, a large portion of observations fell outside the 95% posterior credible intervals. This indicated that the underlying process was more dispersed and dynamic than what those simpler models could accommodate.

Having now explored Bayesian regression to understand time-dependent behavior in the log-return framework, we move to a more flexible approach for modeling trade counts. In this section, we introduce a **hierarchical Bayesian model** based on a Negative Binomial structure, which naturally handles overdispersion and allows parameters to vary across time while still sharing information through higher-level priors. This added flexibility enables the model to capture both the trend and the depth of variability that earlier models could not.

Notation

- i : An index for each observation (each minute of data).
- y_i : The observed `trade_count` for observation i .
- h_i : The hour of the day for observation i , where $h_i \in \{0, 1, \dots, 23\}$.
- $\mathbf{x}_i$: The vector of 5 feature values for observation i .
 - $x_{i,1}$: `lag1_abs_log_return`
 - $x_{i,2}$: `lag1_trade_count`
 - $x_{i,3}$: `lag1_log_volume`
 - $x_{i,4}$: `lag1_buy_sell_imbalance`
 - $x_{i,5}$: `volatility`
- β : The vector of 5 regression coefficients, corresponding to the features.
- α_h : A specific intercept for each hour h .
- μ_i : The expected (mean) `trade_count` for observation i .
- ϕ : A dispersion parameter for the Negative Binomial distribution.
- $\mu_\alpha, \sigma_\alpha$: Parameters governing the distribution of the hourly intercepts.

Hierarchical Model Description

We build a Hierarchical Bayesian Negative Binomial Regression Model that predicts minute-level trade counts using the specified features while accommodating systematic variation across hours of the day.

Layer 1: The Data Likelihood

This layer describes the data-generating process. The model assumes that the `trade_count`, y_i , which is a form of count data, follows a **Negative Binomial distribution**. This choice corrects the flaw of using a **Poisson distribution** which assumes mean = variance as it can model **overdispersion**, which is a common characteristic of financial trade counts.

The distribution for each observation y_i is defined by its mean μ_i and a single, shared dispersion parameter ϕ .

$$y_i \sim \text{NegativeBinomial}(\text{mean} = \mu_i, \text{dispersion} = \phi)$$

where the mean is $\mathbb{E}[y_i] = \mu_i$ and the variance is

$$\text{Var}(y_i) = \mu_i + \frac{\mu_i^2}{\phi}.$$

Here, $\phi > 0$ controls the amount of overdispersion: smaller values allow for larger variance relative to the mean. The mean μ_i itself is determined by the regression components introduced in the next layer.

Layer 2: The Linear Predictor and Link Function

This layer connects the features and hourly effects to the mean trade count μ_i . Since the mean must be positive, the model uses a **log function**. This means the model predicts the logarithm of the mean as a linear combination of the parameters.

The equation for the log of the mean is:

$$\log(\mu_i) = \alpha_{h_i} + \mathbf{x}_i \cdot \boldsymbol{\beta}$$

$$\log(\mu_i) = \alpha_{h_i} + (\beta_1 \cdot x_{i,1}) + (\beta_2 \cdot x_{i,2}) + (\beta_3 \cdot x_{i,3}) + (\beta_4 \cdot x_{i,4}) + (\beta_5 \cdot x_{i,5})$$

$$\begin{aligned} \log(\mu_i) = \alpha_{h_i} &+ \beta_1 \cdot (\text{lag1_abs_log_return}_i) \\ &+ \beta_2 \cdot (\text{lag1_trade_count}_i) \\ &+ \beta_3 \cdot (\text{lag1_log_volume}_i) \\ &+ \beta_4 \cdot (\text{lag1_buy_sell_imbalance}_i) \\ &+ \beta_5 \cdot (\text{volatility}_i) \end{aligned}$$

- α_{h_i} : This is the baseline log-trade-count specific to the hour when observation i occurred.
- $\beta_1, \dots, \beta_5$: These are the coefficients that quantify the effect of each of our features on the log-trade-count.

Layer 3: Priors

This layer specifies our initial priors about the parameters from the layers above before the model sees any data.

- **Regression Coefficients (β):** Each coefficient β_j is given a broad, weakly informative Normal prior centered at zero. This acts as a form of regularization, slightly discouraging extremely large coefficient values.

$$\beta_j \sim \mathcal{N}(\text{mean} = 0, \text{variance} = 10^2) \quad \text{for } j = 1, \dots, 5$$

- **Dispersion Parameter (ϕ):** The dispersion parameter ϕ must be positive. It is given an Exponential prior, which is a common choice for positive-valued scale parameters.

$$\phi \sim \text{Exponential}(\text{rate} = 1)$$

Layer 4: The Hierarchical Structure and Hyperpriors

This is the "hierarchical" part of the model. Instead of treating the 24 hourly intercepts ($\alpha_0, \dots, \alpha_{23}$) as completely independent, the model assumes they are drawn from a common group distribution. This allows the model to borrow strength across hours, leading to more stable estimates, especially for hours with less data.

- **Prior on Hourly Intercepts (α_h):** Each of the 24 hourly intercepts is modeled as being drawn from a single Normal distribution, which is itself defined by a mean μ_α and a standard deviation σ_α .

$$\alpha_h \sim \mathcal{N}(\text{mean} = \mu_\alpha, \text{variance} = \sigma_\alpha^2) \quad \text{for } h = 0, \dots, 23$$

The parameters of this group distribution, μ_α and σ_α , are called **hyperparameters**, and they are also given priors.

- **Hyperprior on the Group Mean (μ_α):** The average hourly effect is given a broad Normal prior.

$$\mu_\alpha \sim \mathcal{N}(\text{mean} = 0, \text{variance} = 10^2)$$

- **Hyperprior on the Group Standard Deviation (σ_α):** The variability between hours, σ_α , must be positive and is given an Exponential prior.

$$\sigma_\alpha \sim \text{Exponential}(\text{rate} = 1)$$

Implementing - MH

Due to the model's complex hierarchical structure, the posterior distribution of the parameters cannot be solved analytically. Therefore, we employ a Markov Chain Monte Carlo (MCMC) method to approximate it. The model is fitted using a Metropolis-Hastings sampler, which is designed to explore the parameter space and draw samples from the target posterior distribution.

We start by initializing all parameters, the sampler iterates for thousands of steps. In each step, it cycles through every parameter from the regression coefficients (β_i) to the hierarchical parameters ($\mu_\alpha, \sigma_\alpha$) and performs a Metropolis update. This involves:

1. Proposing a new value for the parameter,
2. Evaluating the posterior and prior plausibility,
3. Accepting or Rejecting the proposal based on this comparison.

As an initial "burn-in" period, our implementation has discarded the first 2000 iterations to ensure the samples are drawn from the stable, high-probability region of the posterior.

The code for the above model with MH is as follows:

```
class HierarchicalTradeModel:
    def __init__(self, data: pd.DataFrame, features: list, target: str = 'trade_count'):
        if 'timestamp' not in data.columns:
            raise ValueError("Input data must contain a 'timestamp' column.")
        self.data = data.copy()
        self.features = features
        self.target = target
        self.params = None
        self._prepare_data()

    def _prepare_data(self):
        self.data['timestamp'] = pd.to_datetime(self.data['timestamp'])
        self.data['hour'] = self.data['timestamp'].dt.hour
        self.y = self.data[self.target].values
        self.scaler = StandardScaler()
        self.X = self.scaler.fit_transform(self.data[self.features].values)
        self.n_obs, self.n_features = self.X.shape
        self.n_hours = 24
        self.hour_indices = {h: np.where(self.data['hour'] == h)[0] for h in range(self.n_hours)}

    def _log_likelihood(self, y, mu, phi):
        mu = np.maximum(mu, 1e-9)
        phi = np.maximum(phi, 1e-9)
        n = phi
        p = phi / (phi + mu)
        return np.sum(nbinom.logpmf(y, n=n, p=p))

    def _log_priors(self, alphas, betas, phi, mu_alpha, sigma_alpha):
        log_p = 0
        log_p += np.sum(norm.logpdf(alphas, loc=mu_alpha, scale=sigma_alpha))
        log_p += np.sum(norm.logpdf(betas, loc=0, scale=10))
        log_p += expon.logpdf(phi, scale=1)
        log_p += norm.logpdf(mu_alpha, loc=0, scale=10)
        log_p += expon.logpdf(sigma_alpha, scale=1)
        return log_p

    def compute_posterior(self, n_samples: int, burn_in: int, proposal_widths: dict):
        params_current = {
            'alphas': np.zeros(self.n_hours), 'betas': np.zeros(self.n_features),
            'phi': 1.0, 'mu_alpha': 0.0, 'sigma_alpha': 1.0
        }
        samples = {key: [] for key in params_current}
        print("Starting MCMC sampling...")
        for i in tqdm(range(n_samples + burn_in)):
            for j in range(self.n_features):
                params_current['betas'][j], _ = self._metropolis_step(
                    'betas', j, params_current, proposal_widths.get('betas', 0.1))
            for h in range(self.n_hours):
                params_current['alphas'][h], _ = self._metropolis_step(
```

```

        'alphas', h, params_current, proposal_widths.get('alphas', 0.1))
    params_current['phi'], _ = self._metropolis_step(
        'phi', None, params_current, proposal_widths.get('phi', 0.1))
    params_current['mu_alpha'], _ = self._metropolis_step(
        'mu_alpha', None, params_current, proposal_widths.get('mu_alpha', 0.1))
    params_current['sigma_alpha'], _ = self._metropolis_step(
        'sigma_alpha', None, params_current, proposal_widths.get('sigma_alpha', 0.1))
    if i >= burn_in:
        for key, value in params_current.items():
            samples[key].append(np.copy(value))
    self.params = {key: np.array(value) for key, value in samples.items()}

def _metropolis_step(self, param_name, param_index, params_current, proposal_width):
    params_proposal = {k: np.copy(v) for k, v in params_current.items()}
    current_value = params_current[param_name] if param_index is None else params_current[param_name][param_index]
    proposal = current_value + norm.rvs(scale=proposal_width)
    if param_name in ['phi', 'sigma_alpha'] and proposal <= 0: return current_value, 0
    if param_index is None: params_proposal[param_name] = proposal
    else: params_proposal[param_name][param_index] = proposal
    log_post_current = self._calculate_log_posterior(params_current)
    log_post_proposal = self._calculate_log_posterior(params_proposal)
    if np.log(np.random.rand()) < log_post_proposal - log_post_current:
        return proposal, 1
    else:
        return current_value, 0

def _calculate_log_posterior(self, params):
    alphas, betas, phi = params['alphas'], params['betas'], params['phi']
    mu_alpha, sigma_alpha = params['mu_alpha'], params['sigma_alpha']
    log_mu = self.X @ betas
    for h in range(self.n_hours):
        log_mu[self.hour_indices[h]] += alphas[h]
    mu = np.exp(log_mu)
    return self._log_likelihood(self.y, mu, phi) + self._log_priors(alphas, betas, phi, mu_alpha, sigma_alpha)

def _generate_posterior_predictive_samples(self):
    if self.params is None:
        raise RuntimeError("Posterior has not been computed. Please run compute_posterior() first.")

    num_posterior_samples = len(self.params['phi'])
    y_pred_samples = np.zeros((num_posterior_samples, self.n_obs))

    for i in tqdm(range(num_posterior_samples), desc="Generating predictive samples"):
        alphas_i = self.params['alphas'][i]
        betas_i = self.params['betas'][i]
        phi_i = np.maximum(self.params['phi'][i], 1e-9)

        log_mu_i = self.X @ betas_i + alphas_i[self.data['hour'].values]
        mu_i = np.exp(log_mu_i)
        mu_i = np.maximum(mu_i, 1e-9)

        n = phi_i
        p = phi_i / (phi_i + mu_i)

        y_pred_samples[i, :] = nbinom.rvs(n=n, p=p, size=self.n_obs)

    return y_pred_samples

def calculate_posterior_predictive_coverage(self, credible_interval: float = 95.0):
    y_pred_samples = self._generate_posterior_predictive_samples()

    # Calculating lower and upper bounds for the credible interval
    lower_bound = np.percentile(y_pred_samples, (100 - credible_interval) / 2, axis=0)
    upper_bound = np.percentile(y_pred_samples, 100 - (100 - credible_interval) / 2, axis=0)

```

```

# Checking which actual data points fall outside the interval
outliers = (self.y < lower_bound) | (self.y > upper_bound)
num_outliers = np.sum(outliers)

coverage_percentage = (num_outliers / self.n_obs) * 100

print("\n--- Posterior Predictive Coverage ---")
print(f"Expected percentage of data outside {credible_interval}% credible interval: {100 - credible_interval:.2f}%")
print(f"Actual percentage of data outside {credible_interval}% credible interval: {coverage_percentage:.2f}%")

return coverage_percentage

def plot_posterior_predictive(self, credible_interval: float = 95.0, sample_size: int = None):
    y_pred_samples = self._generate_posterior_predictive_samples()

    if sample_size is not None and sample_size < self.n_obs:
        plot_indices = np.linspace(0, self.n_obs - 1, sample_size, dtype=int)
    else:
        plot_indices = np.arange(self.n_obs)

    y_plot = self.y[plot_indices]
    timestamps_plot = self.data['timestamp'].iloc[plot_indices]
    y_pred_plot = y_pred_samples[:, plot_indices]

    mean_prediction = np.mean(y_pred_plot, axis=0)
    lower_bound = np.percentile(y_pred_plot, (100 - credible_interval) / 2, axis=0)
    upper_bound = np.percentile(y_pred_plot, 100 - (100 - credible_interval) / 2, axis=0)

    print("Plotting results...")
    plt.style.use('seaborn-v0_8-whitegrid')
    fig, ax = plt.subplots(figsize=(18, 8))

    ax.plot(timestamps_plot, y_plot, linestyle="--", color='darkorange', linewidth=1.5,
            alpha=0.8, label='Actual Trade Count')

    ax.plot(timestamps_plot, mean_prediction, color='#003f5c', lw=2, label='Posterior Mean Prediction')
    ax.fill_between(timestamps_plot, lower_bound, upper_bound, color='#4f98ca', alpha=0.4,
                    label=f'{credible_interval}% Predictive Interval')

    # Using a symmetric log scale to handle extreme outliers and zeros.
    ax.set_yscale('symlog', linthresh=1)

    ax.set_title('Posterior Predictive Check (Log Scale)', fontsize=18)
    ax.set_xlabel('Timestamp', fontsize=12)
    ax.set_ylabel('Number of Trades per Minute (Log Scale)', fontsize=12)
    ax.legend(loc='upper left')

    ax.grid(True, which='major', linestyle='-', linewidth='0.5', color='gray')
    ax.grid(True, which='minor', linestyle=':', linewidth='0.5', color='lightgray')

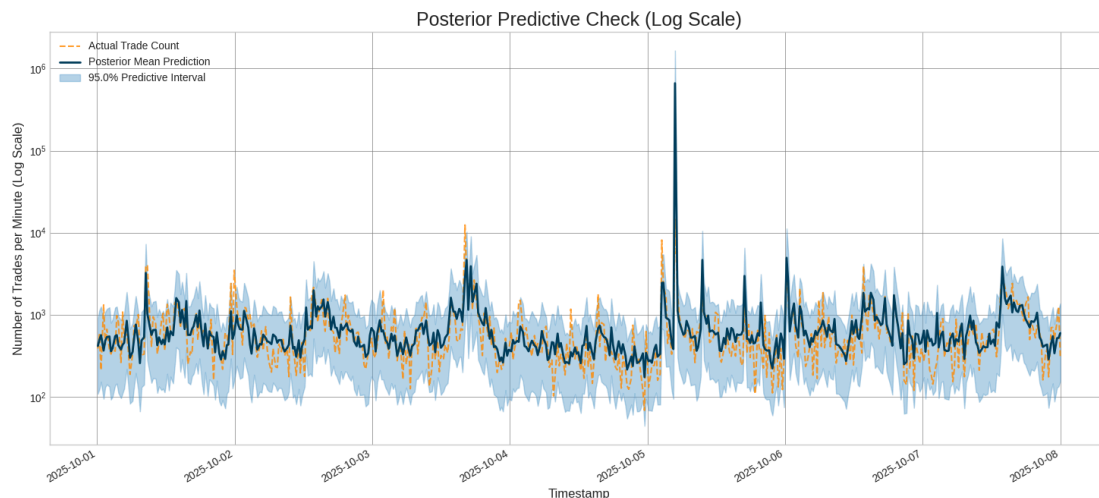
    fig.autofmt_xdate()
    filename = f"Hierarchical_PPC.png"
    file_path = os.path.join("../figures/", filename)
    plt.savefig(file_path, format='png')
    plt.show()

```

To evaluate the model's fit, we compare the **actual trade counts** (orange dashed line) with the **posterior mean prediction** (dark blue line) and the **95% credible predictive interval** (blue shaded region). The credible interval shows the range where the model expects 95% of future observations to fall, capturing its predictive uncertainty. A symmetric log scale is used so that both normal activity and extreme spikes can be viewed clearly without distortion. And the following code is used to plot the above evaluation.

```
# features_to_use = ['log_volume', 'buy_sell_imbalance', 'volatility']
model = HierarchicalTradeModel(data=df, features=features_col, target='trade_count')

proposal_widths = {
    'betas': 0.05, 'alphas': 0.1, 'phi': 0.2,
    'mu_alpha': 0.1, 'sigma_alpha': 0.1
}
model.compute_posterior(n_samples=5000, burn_in=2000, proposal_widths=proposal_widths)
if model.params:
    model.plot_posterior_predictive(sample_size=500)
    model.calculate_posterior_predictive_coverage(credible_interval=95.0)
```



Visually, the model seems to fit the data very well. The posterior mean closely follows the main trends and daily patterns in the trade counts, and almost all observations, including large spikes, fall within the 95% credible interval. This shows that the model not only captures the average behavior but also provides realistic uncertainty estimates that account for the high volatility in the data.

This visual impression is supported by the coverage statistics: with a 95% interval, we expect about 5% of observations to fall outside the band. The model produces **4.13%** outliers, very close to the theoretical target, indicating that it is well-calibrated. It is neither too confident nor too uncertain.

Overall, the posterior predictive check shows that this hierarchical model fits the trade count data strongly, captures key patterns, and produces reliable uncertainty estimates suitable for understanding and predicting market activity.

Conclusion

In this project we have successfully expanded the initial Bayesian analysis of BTC/USDT trade data by introducing stronger modeling tools and more formal methods for inference. Through Bayes factor analysis under simple and composite hypotheses, we showed that the long-run mean of the log-trade volume process is tightly concentrated near 14, while short-term inferences remain highly unstable and sensitive to data size.

Bayesian regression provided a structured way to measure how market features influence log-returns and trade counts. Although return volatility remains difficult to predict, the model's credible intervals captured nearly all observations, demonstrating reliable uncertainty estimation.

The most substantial improvement came from the **Hierarchical Negative Binomial Regression Model**, which addressed overdispersion and daily seasonality. And Metropolis–Hastings sampler successfully fit the model, and posterior predictive checks showed excellent calibration: the 95% interval captured 95.87% of observations, almost exactly matching theoretical expectations.

Overall, this work shows that advanced Bayesian tools such as MCMC, hierarchical modeling, and careful uncertainty quantification offer a powerful framework for capturing the complex behavior of cryptocurrency markets and form a strong foundation for future predictive and anomaly-detection applications.